Task 1

Part 1

Default

```
The sum of the first 5 natural numbers is 15
The 7th Fibonacci number is 13
The factorial of 5 is 120
```

All set to 10

```
The sum of the first 10 natural numbers is 55
The 10th Fibonacci number is 55
The factorial of 10 is 3628800
```

Set to 1

```
The sum of the first 1 natural numbers is 1
The 1th Fibonacci number is 1
The factorial of 1 is 1
```

Part 2

```python
# Iterative version
def sum_of_natural_numbers_iteration(n):
    total = 0
    for _ in range(1, n + 1):
        total += _
    return total
```

```python
# Iterative version
def fibonacci_iteration(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1

    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

```python
def factorial(n):
    if n == 0 or n == 1:
        return 1  # Base case
    else:
        return n * factorial(n - 1)  # Recursive case

def factorial_iteration(n):
    result = 1
    for _ in range(2, n + 1):
        result *= _
    return result
```

Part 3

Default

```
SUM OF NATURAL NUMBERS
Recursive: 0.000036
Iterative: 0.000020
FACTORIAL
Recursive: 0.000053
Iterative: 0.000024
FIBONACCI
Recursive: 0.003313
Iterative: 0.000026
```

30

```
SUM OF NATURAL NUMBERS
Recursive: 0.000048
Iterative: 0.000024
FACTORIAL
Recursive: 0.000071
Iterative: 0.000033
FIBONACCI
Recursive: 4.325981
Iterative: 0.000031
```

35

```
SUM OF NATURAL NUMBERS
Recursive: 0.000055
Iterative: 0.000027
FACTORIAL
Recursive: 0.000083
Iterative: 0.000038
FIBONACCI
Recursive: 48.216282
Iterative: 0.000035
```

Above are a number of trials all time based that were undertaken in order to enable a benchmark for comparison between recursive and iterative style. It is important to note that the SONN and Factorial tests do not greatly differ in time as the data sets of 20,30 and 35 are really small. However, when it comes to the fibonacci sequence 35 is relatively high and can become time consuming, particularly recursive. This is due to the exponential growth seen in recursive calls, which happens due to the fact that each subproblem is continuously recalculated. For example, due the way in which recursive work the Fibonacci ends up resulting in a time complexity of 0(2^n), this means a data set of 80 up until recently (quantum computing) would take longer than the age of the universe. In comparison, irritative only completes each step once and as such we end up with a far nicer complexity of 0(n)

Task 2

Task 1 - Merge

Default

```
[2, 6, 9, 5, 3, 4, 5, 6, 6, 7]

[2, 3, 4, 5, 5, 6, 6, 6, 7, 9]
[1]
[2, 30, 31, 31, 43, 53, 56, 68, 91, 98]
3240
```

Changed List, Range (1, 5000) for n in range(100)] with 70 Elements

```
[2, 6, 9, 5, 3, 4, 5, 6, 6, 7, 7, 100]

[2, 3, 4, 5, 5, 6, 6, 6, 7, 7, 9, 100]
[1]
[16, 20, 96, 159, 173, 209, 293, 313, 387, 456, 493, 509, 568, 606, 654, 724, 735, 736, 747, 768, 775, 829, 833, 842, 847, 857, 874, 934, 951, 974, 975, 1103, 1167, 1168, 1227, 1259, 1341, 13
97, 1404, 1499, 1505, 1604, 1653, 1662, 1696, 1801, 1846, 1852, 1881, 1903, 2020, 2022, 2031, 2052, 2072, 2123, 2146, 2156, 2166, 2249, 2304, 2543, 2577, 2611, 2654, 2708, 2831, 2992, 3084, 3
103, 3118, 3178, 3235, 3257, 3273, 3316, 3378, 3443, 3481, 3662, 3735, 3744, 3776, 3778, 3780, 3814, 3826, 4023, 4035, 4145, 4154, 4154, 4213, 4214, 4233, 4364, 4631, 4737, 4749, 4862]
```

Changed List, Range (1,25) for n in range (25) with 25 elements

```
[2, 6, 9, 5, 3, 4, 5, 6, 6, 7, 7, 100, 1, 4]

[1, 2, 3, 4, 4, 5, 5, 6, 6, 6, 7, 7, 9, 100]
[1]
[4, 4, 4, 4, 5, 5, 6, 7, 8, 9, 10, 13, 14, 15, 16, 17, 18, 18, 18, 19, 20, 20, 22, 24, 25]
```

Task 2 - Insertion

Changed List, Range (1,25) for n in range (25) with 25 elements

```
[1, 2, 3, 4, 4, 5, 5, 6, 6, 6, 7, 7, 9, 100]
[1]
[1, 3, 3, 6, 7, 11, 13, 13, 13, 14, 15, 15, 15, 16, 16, 17, 18, 19, 19, 20, 20, 21, 21, 22, 25
```

Changed List, Range (1,300) for n in range (15) with 15 elements

```
[1, 2, 2.5, 3, 4, 4, 5, 5, 6, 6, 6, 7, 7, 9, 100, 600]
[1]
[24, 31, 61, 62, 86, 125, 127, 156, 164, 188, 198, 210, 241, 244, 295]
```

BenchMarking

Merge

Default

```
Insertion Sort took: 0.002679 seconds
Merge Sort took: 0.001945 seconds
[2, 6, 9, 5, 3, 4, 5, 6, 6, 7]

[2, 3, 4, 5, 5, 6, 6, 6, 7, 9]
[1]
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 5, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6]
3240
```

99999

```
Merge Sort took: 3.663735 seconds
```

999999

```
Merge Sort took: 49.131314 seconds
```

Insertion

10000

```
Insertion Sort took: 17.131447 seconds
Merge Sort took: 0.191938 seconds
[2, 6, 9, 5, 3, 4, 5, 6, 6, 7]

[2, 3, 4, 5, 5, 6, 6, 6, 7, 9]
[1]
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 5, 5, 5, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 6, 6]
3240
```

5000

```
Insertion Sort took: 6.954700 seconds
```

2500

```
Insertion Sort took: 1.747003 seconds
```

As can be clearly seen insertion sort takes far longer than merge sort. Once again similar to the above this can be put down to the different complexity levels of each different sorting function. Merge sort uses a divide and conquer technique which relies on the complexity of 0(nlogn) while insertion uses 0(n^2) (similar to Fibonacci). As a result merge scales more efficiently with higher data sets then insertion as demonstrated above.

Task 3

Part 1

Disk Size 5

```
Enter number of disks: 5
The moves are:
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
Move disk 3 from A to B
Move disk 1 from C to A
Move disk 2 from C to B
Move disk 1 from A to B
Move disk 4 from A to C
Move disk 1 from B to C
Move disk 2 from B to A
Move disk 1 from C to A
Move disk 3 from B to C
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
Move disk 5 from A to B
Move disk 1 from C to A
Move disk 2 from C to B
Move disk 1 from A to B
Move disk 3 from C to A
Move disk 1 from B to C
Move disk 2 from B to A
Move disk 1 from C to A
Move disk 4 from C to B
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
Move disk 3 from A to B
Move disk 1 from C to A
Move disk 2 from C to B
Move disk 1 from A to B
```

Disk Size 3

```
Enter number of disks: 3
The moves are:
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
Move disk 3 from A to B
Move disk 1 from C to A
Move disk 2 from C to B
Move disk 1 from A to B
```

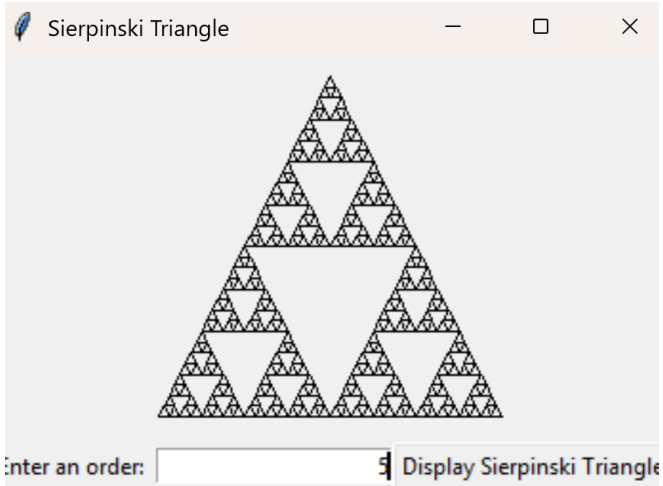
Part 2

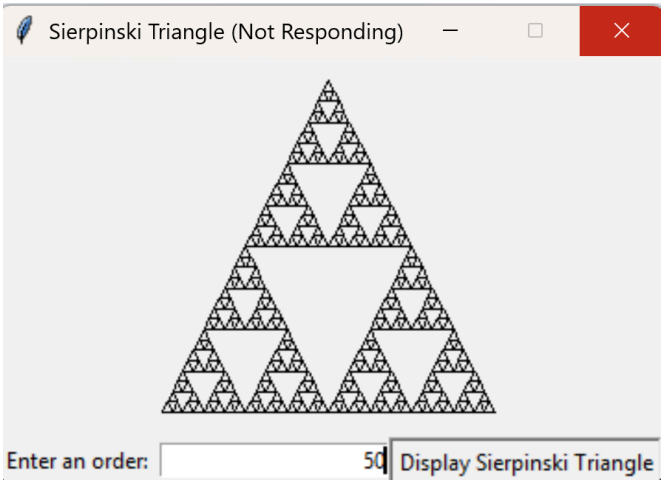
```python
move_count = 0

def main1():
    global move_count
    n = eval(input("Enter number of disks: "))
    move_count = 0
    print("The moves are:")
    moveDisks(n, 'A', 'B', 'C')
    print("Number of moves is", move_count)

def moveDisks(n, fromTower, toTower, auxTower):
    global move_count
    if n == 1:
        move_count += 1
    else:
        moveDisks(n - 1, fromTower, auxTower, toTower)
        move_count += 1
        moveDisks(n - 1, auxTower, toTower, fromTower)

main1()
```

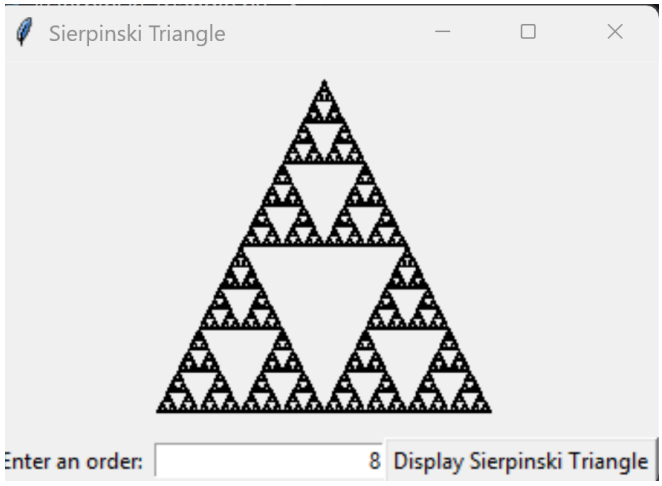
```
Enter number of disks: 8
The moves are:
Number of moves is 255
```

Task 4

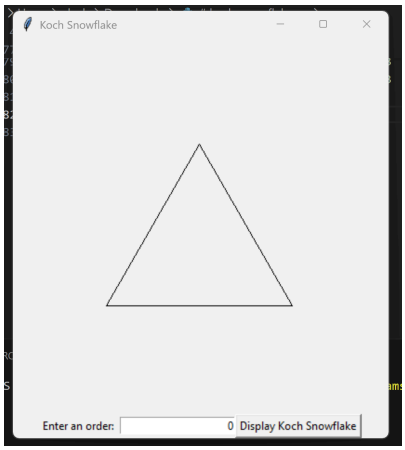
Part 1

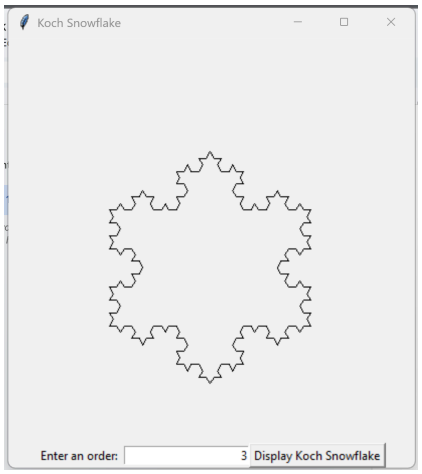

Sierpinski Triangle
Enter an order:
5
Display Sierpinski Triangle

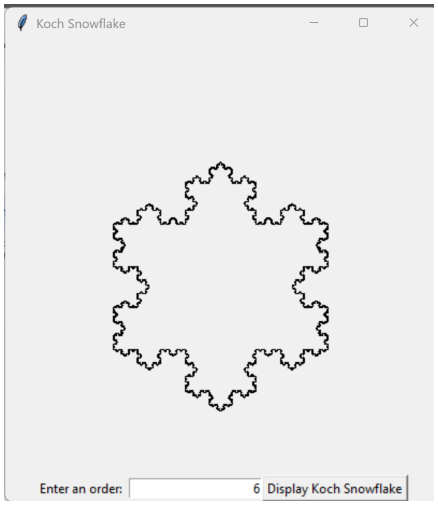

Sierpinski Triangle (Not Responding)
Enter an order:
50
Display Sierpinski Triangle


Sierpinski Triangle
Enter an order:
8
Display Sierpinski Triangle

Part 2


Koch Snowflake
Enter an order:
0
Display Koch Snowflake


Koch Snowflake
Enter an order:
3
Display Koch Snowflake


Koch Snowflake
Enter an order:
6
Display Koch Snowflake